Laboratorium Algorytmów Macierzowych - ćwiczenie 1

ćwiczenie 1 - Mnożenie Macierzy

Autor: Artur Radwański

Kraków
Semestr letni 2026



Wydział Informatyki
Akademia Górniczo-Hutnicza

1 Środowisko

- Procesor: AMD Ryzen 7 9800X3D
- Pamięć: 64GB 6000MT/s
- Język: C++26
- Kompilator: gcc version 16.2.1 20260810 (GCC)
- System Operacyjny: 7.2.10-arch1-1

Język c++ jest językiem kompilowalnym. Kompilator tłumaczy kod źródłowy do języka mszynowego.

2 Algorytm Bineta

Algorithm 1 Rekurencyjny algorytm mnożenia macierzy (Binet / Dziel i Zwyciężaj)

Require: Macierze A, B o wymiarach $N \times N$, wyzerowana macierz wynikowa C
Ensure: Zaktualizowana macierz $C = C + A \times B$

function BinetMultiply(A, B, C)
  if $N \leq N_{base}$ **then**
    NaiveMultiply(A, B, C, N) ▷ Klasyczne 3 pętle for
    return
  end if
  $A_{11}, A_{12}, A_{21}, A_{22} \leftarrow$Split(A)
  $B_{11}, B_{12}, B_{21}, B_{22} \leftarrow$Split(B)
  $C_{11}, C_{12}, C_{21}, C_{22} \leftarrow$Split(C)
  BinetMultiply(A_{11}, B_{11}, C_{11})
  BinetMultiply(A_{12}, B_{21}, C_{11})
  BinetMultiply(A_{11}, B_{12}, C_{12})
  BinetMultiply(A_{12}, B_{22}, C_{12})
  BinetMultiply(A_{21}, B_{11}, C_{21})
  BinetMultiply(A_{22}, B_{21}, C_{21})
  BinetMultiply(A_{21}, B_{12}, C_{22})
  BinetMultiply(A_{22}, B_{22}, C_{22})
end function

Algorytm został zmodyfikowany, aby mógł również obsługiwać macierze prostokątne. Macierze są dzielone zamiast na 2 bloki wzdłuż ich najdłuższego wymiaru zamiast na 4. Dla macierzy kwadratowych 3 kroki rekurencyjne modyfikacji są równoważne 1 krokowi rekurencji oryginalnego algorytmu. Dzięki temu Algorytm nie ma problemu z macierzami których wymiar nie jest potęgą 2.

Algorytm ten ma złożoność $O(n^3)$. Na każdym poziomie rekurencji wykonujemy 8 mnożeń, co nie przekłada się na jakikolwiek zysk względem naiwnego podejścia iteracyjnego.

3 Algorytm Strassena

Algorytm strassena wykonuje jedynie 7 mnożeń na jednym poziomie rekurencji, co poprawia jego złożoność asymptotyczną. Do $O(n^{\log_2 7})$.

Algorithm 2 Algorytm Strassena

Require: Macierze A, B o wymiarach $N \times N$, wyzerowana macierz wynikowa C
Ensure: Zaktualizowana macierz $C = C + A \times B$

function StrassenMultiply(A, B)
  $N \leftarrow$ Length(A)
  if $N \leq N_{base}$ **then**
    return NaiveMultiply(A, B)
  end if
  $A_{11}, A_{12}, A_{21}, A_{22} \leftarrow$Split(A)
  $B_{11}, B_{12}, B_{21}, B_{22} \leftarrow$Split(B)
  $C_{11}, C_{12}, C_{21}, C_{22} \leftarrow$Split(C)
  $M_1 \leftarrow$ StrassenMultiply$(A_{11} + A_{22}, B_{11} + B_{22})$

  $M_2 \leftarrow$ StrassenMultiply$(A_{21} + A_{22}, B_{11})$

  $M_3 \leftarrow$ StrassenMultiply$(A_{11}, B_{12} - B_{22})$

  $M_4 \leftarrow$ StrassenMultiply$(A_{22}, B_{21} - B_{11})$

  $M_5 \leftarrow$ StrassenMultiply$(A_{11} + A_{12}, B_{22})$

  $M_6 \leftarrow$ StrassenMultiply$(A_{21} - A_{11}, B_{11} + B_{12})$

  $M_7 \leftarrow$ StrassenMultiply$(A_{12} - A_{22}, B_{21} + B_{22})$
  $C_{11} \leftarrow C_{11} + M_1 + M_4 - M_5 + M_7$
  $C_{12} \leftarrow C_{12} + M_3 + M_5$
  $C_{21} \leftarrow C_{21} + M_2 + M_4$
  $C_{22} \leftarrow C_{22} + M_1 - M_2 + M_3 + M_6$
  return C
end function

3.1 Peelieng

Aby algorytm działał dla macierzy kwadratowych, w krokach rekurencji gdzie podmacierze mają nieparzyste wymiary zastosowano Peeling

$$A = \begin{pmatrix} A' & u \\ v^T & \alpha \end{pmatrix}, \quad B = \begin{pmatrix} B' & w \\ z^T & \beta \end{pmatrix}$$

Gdzie:

- A', B' to główne podmacierze o wymiarach $2k \times 2k$,
- u, w to wektory kolumnowe o rozmiarze $2k \times 1$,
- v^T, z^T to wektory wierszowe o rozmiarze $1 \times 2k$,
- α, β to pojedyncze wartości skalarne (1×1).

Wynikowa macierz $C = A \cdot B$ ma postać:

$$C = \begin{pmatrix} C' & x \\ y^T & \gamma \end{pmatrix}$$

Poszczególne składowe obliczane są w następujący sposób:

$$C' = \text{Strassen}(A', B') + u \cdot z^T \quad \text{(Podmacierz o parzystych wymiarach)}$$
$$x = A' \cdot w + \beta \cdot u \quad \text{(Ostatnia kolumna)}$$
$$y^T = v^T \cdot B' + \alpha \cdot z^T \quad \text{(Ostatni wiersz)}$$
$$\gamma = v^T \cdot w + \alpha \cdot \beta \quad \text{(Narożny skalar)}$$

Obliczenie dodatkowego wiersza oraz kolumny poza głównym algorytmem nie wpływa na złożoność asymptotyczną, ponieważ złożoność tej operacji to $O(n^2)$, zatem zwiększa ona jedynie narzut stały algorytmu.

4 Algorytm Ladermana

Algorithm 3 Algorytm Ladermana

Require: Macierze A, B o wymiarach $N \times N$, wyzerowana macierz wynikowa C
Ensure: Zaktualizowana macierz $C = C + A \times B$

function L(A, B)
  if $N \leq N_{base}$ **then**
    return NaiveMultiply(A, B)

  end if
  $M_1 \leftarrow L(A_{11} + A_{12} + A_{13} - A_{21} - A_{22} - A_{32} - A_{33}, B_{22})$
  $M_2 \leftarrow L(A_{11} - A_{21}, B_{22} - B_{12})$
  $M_3 \leftarrow L(A_{22}, B_{12} - B_{11} + B_{21} - B_{22} - B_{23} - B_{31} + B_{33})$
  $M_4 \leftarrow L(A_{21} - A_{11} + A_{22}, B_{11} - B_{12} + B_{22})$
  $M_5 \leftarrow L(A_{21} + A_{22}, B_{12} - B_{11})$
  $M_6 \leftarrow L(A_{11}, B_{11})$
  $M_7 \leftarrow L(A_{31} - A_{11} + A_{32}, B_{11} - B_{13} + B_{23})$
  $M_8 \leftarrow L(A_{31} - A_{11}, B_{13} - B_{23})$
  $M_9 \leftarrow L(A_{31} + A_{32}, B_{13} - B_{11})$
  $M_{10} \leftarrow L(A_{11} + A_{12} + A_{13} - A_{22} - A_{23} - A_{31} - A_{32}, B_{23})$
  $M_{11} \leftarrow L(A_{32}, B_{13} - B_{11} + B_{21} - B_{22} - B_{23} - B_{31} + B_{32})$
  $M_{12} \leftarrow L(A_{32} - A_{13} + A_{33}, B_{22} + B_{31} - B_{32})$
  $M_{13} \leftarrow L(A_{13} - A_{33}, B_{22} - B_{32})$
  $M_{14} \leftarrow L(A_{13}, B_{31})$
  $M_{15} \leftarrow L(A_{32} + A_{33}, B_{32} - B_{31})$
  $M_{16} \leftarrow L(A_{22} - A_{13} + A_{23}, B_{23} + B_{31} - B_{33})$
  $M_{17} \leftarrow L(A_{13} - A_{23}, B_{23} - B_{33})$
  $M_{18} \leftarrow L(A_{22} + A_{23}, B_{33} - B_{21})$
  $M_{19} \leftarrow L(A_{12}, B_{21})$
  $M_{20} \leftarrow L(A_{23}, B_{32})$
  $M_{21} \leftarrow L(A_{21}, B_{13})$
  $M_{22} \leftarrow L(A_{31}, B_{12})$
  $M_{23} \leftarrow L(A_{33}, B_{33})$
  $C_{11} \leftarrow C_{11} + M_6 + M_{14} + M_{19}$
  $C_{12} \leftarrow C_{12} + M_1 + M_4 + M_5 + M_6 + M_{12} + M_{14} + M_{15}$
  $C_{13} \leftarrow C_{13} + M_6 + M_7 + M_9 + M_{10} + M_{14} + M_{16} + M_{18}$
  $C_{21} \leftarrow C_{21} + M_2 + M_3 + M_4 + M_6 + M_{14} + M_{16} + M_{17}$
  $C_{22} \leftarrow C_{22} + M_2 + M_4 + M_5 + M_6 + M_{20}$
  $C_{23} \leftarrow C_{23} + M_{14} + M_{16} + M_{17} + M_{18} + M_{21}$
  $C_{31} \leftarrow C_{31} + M_6 + M_7 + M_8 + M_{11} + M_{12} + M_{13} + M_{14}$
  $C_{32} \leftarrow C_{32} + M_{12} + M_{13} + M_{14} + M_{15} + M_{22}$
  $C_{33} \leftarrow C_{33} + M_6 + M_7 + M_8 + M_9 + M_{23}$
  return C
end function

Algorytm wykonuje jedynie 23 mnożenia zamiast 27 co przekłada się na złożoność asymptotyczną $(n^{\log_3 23})$. Jest to wynik nieco gorszy niż dla algorytmu Strassena, w dodatku algorytm Ladermana alokuje znacznie więcej macierzy pomocniczych oraz wykonuje o wiele więcej dodawań, co przekłada się na gigantyczny narzut stały.

4.1 Peelieng

Algorytm Ladermana wykorzystuje podział 3×3. Jeśli wymiar N nie jest podzielny przez 3, reszta z dzielenia wynosi $r = N \bmod 3 \in \{1, 2\}$. Macierze rozbijane są na główną podmacierz $3k \times 3k$ oraz brzegowe paski o szerokości r:

$$A = \begin{pmatrix} A' & U \\ V^T & \alpha \end{pmatrix}, \quad B = \begin{pmatrix} B' & W \\ Z^T & \beta \end{pmatrix} \tag{1}$$

Gdzie:

- A', B' to podmacierze o wymiarach $3k \times 3k$,
- U, W to paski pionowe o wymiarze $3k \times r$,
- V^T, Z^T to paski poziome o wymiarze $r \times 3k$,
- α, β to bloki narożne o rozmiarze $r \times r$.

Rozwiązanie dla macierzy wynikowej $C = \begin{pmatrix} C' & X \\ Y^T & \Gamma \end{pmatrix}$ wylicza się analogicznie do metody Strassena, uwzględniając szersze wymiary pasków:

$$C' = \text{Laderman}(A', B') + U \cdot Z^T \qquad \text{(Rdzeń } 3k \times 3k + \text{ iloczyn macierzy paskowych)} \tag{2}$$
$$X = A' \cdot W + U \cdot \beta \qquad \text{(Prawy pas } 3k \times r) \tag{3}$$
$$Y^T = V^T \cdot B' + \alpha \cdot Z^T \qquad \text{(Dolny pas } r \times 3k) \tag{4}$$
$$\Gamma = V^T \cdot W + \alpha \cdot \beta \qquad \text{(Narożnik } r \times r) \tag{5}$$

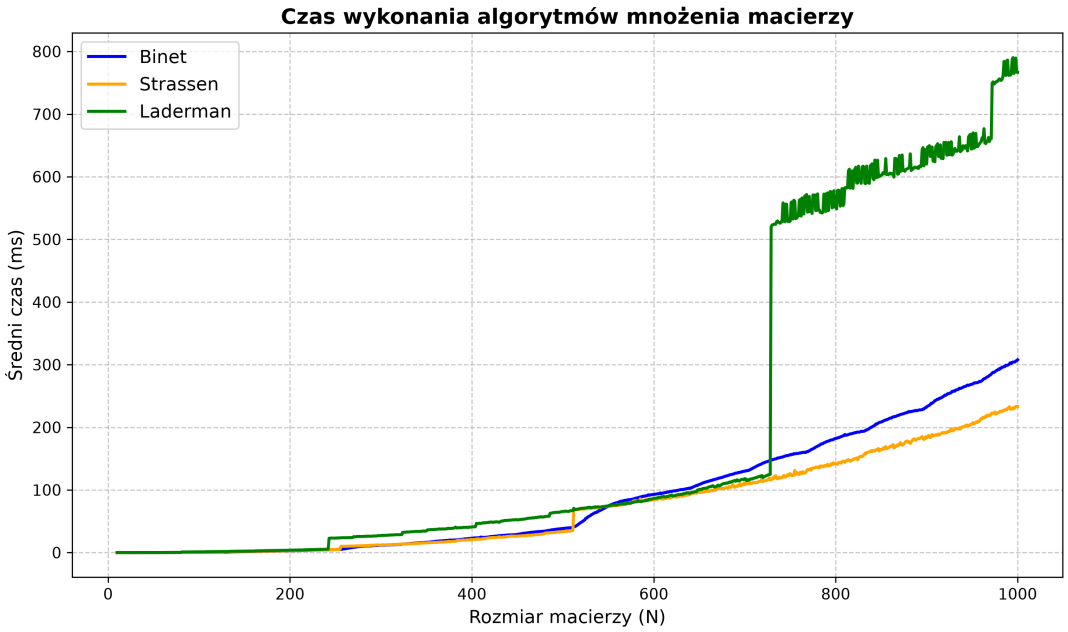
Ponieważ $r < 3$ to operacje przemnożenia pasków zachowują złożoność $O(n^2)$ ze względu na n, peeling nie zmienia więc złożoności obliczeniowej również dla ladermana.

5 Pomiary

Testy wykonywano dla kwadratowych macierzy o wymiarach $n \times n$ gdzie $n \in [10, 1000]$ W komórkach macierzy zapisaono losową wartość $x \in (10^{-8}, 1)$. X były zmiennymi typu Binary64;

5.1 Czas

Aby zmierzyć czas wykonania algorytmu uruchamiano mnożenie 10 razy dla tej samej macierzy, po czym zapisywano uśredniony czas wykonania w milisekundach

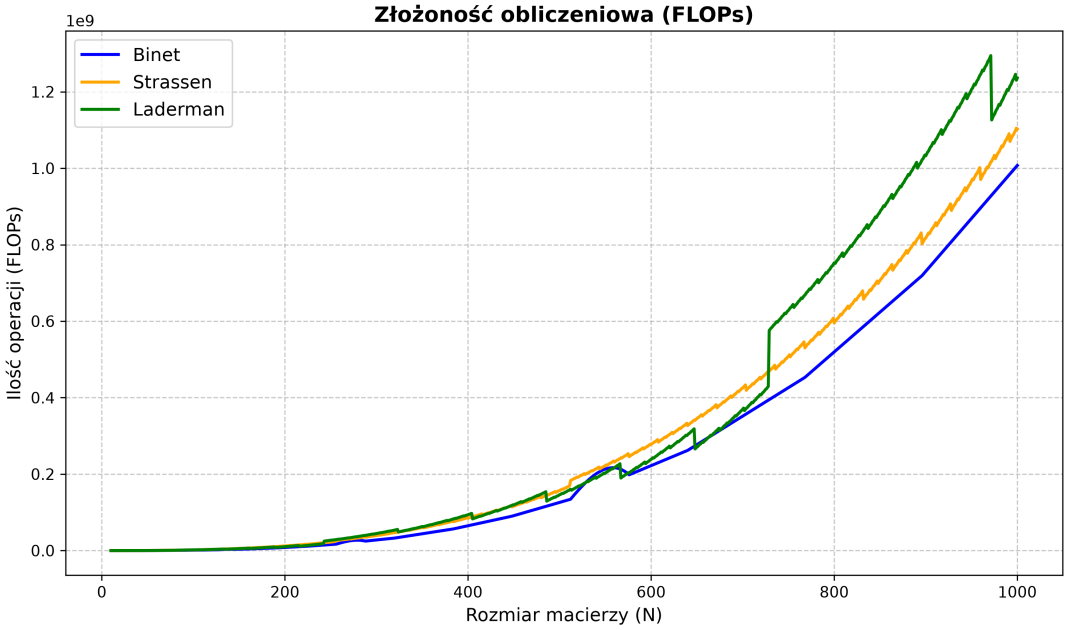


Rysunek 1: Wykres czasu wykonania algorytmów

Jak widać na rysunku 1 dla mniejszych macierzy ($n < 729$) wszystkie algorytmy radziły sobie podobnie. Później jednak algorytm ladermana musi wykonać dodatkowy krok rekurencji, co sprawia, że daje o wiele gorsze wyniki niż pozostałe algorytmy. Narzut z peelingu w przypadku Ladermana jest większy niż dla Strassena, przez co widać gwałtowne skoki na wykresie, dla liczb które nie dzielą się wielokrotnie przez 3. Algorytm Strassena zgodnie z przewidywaniami radzi sobie najlepiej.

5.2 FLOPS

Liczba flops została zliczona w trakcie jednego wykonania algorytmu dla danego rozmiaru macierzy.



Rysunek 2: Wykres flops wykonanych w trakcie algorytmów

Na rysunku widać, że dla $n = 972$ równocześnie mocno spadła liczba flops i wzrósł czas wykonania. Jest to wskazówka, że ten spadek wydajności jest prawdopodobnie raczej powiązany z zarządzaniem pamięcią i

ładowaniem danych do L2 i L1 cache procesora niż z uwarunkowaniami artmetycznymi algorytmu. Flops pozostałych dwóch algorytmów zachowują się w sposób przewidywalny. Pokazują one też, że zamiana dodawań na mnożenia faktycznie przynosi zyski. Algorytm Strassena, mimo że miał więcej flopsów, to był szybszy niż binet.

5.3 Kontrola błędów

Wyniki algorytmów porównano z wynikami profesjonalnej bibiloteki Eigen3 do operacji macierzowych, poniżej podano błąd maksymalny dla każdego algorytmu:

$$n = 4$$

- binet: 0
- strassen: 3.330669e-16
- laderman 2.220446e-16

$$n = 100$$

- binet: 0
- Strassen: 7.425172e-13
- Laderman: 4.263256e-14

Jak widać wszystkie wyniki mieszczą się w zakresie błędu zmiennoprzecinkowego co potwierdza ich poprawność